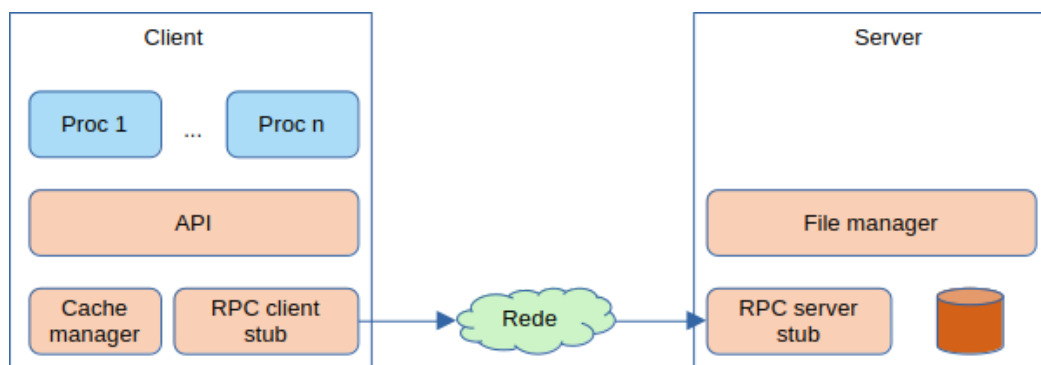


Definição do Trabalho 1: Sistemas de Arquivos Remoto

Sistemas de arquivo remotos permitem a persistência de dados em um servidor de acesso por múltiplos clientes, favorecendo o compartilhamento de arquivos. Em alguns casos, sistemas de arquivo remotos podem oferecer escalabilidade ao utilizar particionamento do espaço de armazenamento, segurança e tolerância a falhas.

Neste trabalho, deve ser implementado um sistema de arquivos remoto simplificado. O objetivo é que múltiplos clientes possam escrever ou ler arquivos em um servidor remoto. Requisitos de segurança, escalabilidade e tolerância a falhas serão omitidos neste estágio do trabalho. Porém, uma etapa consiste no uso de padrões de projeto para computação distribuída e o grupo poderá usar algum padrão visando melhorar algum desses atributos.

A figura a seguir mostra um cliente e servidor, sendo que processos executados no cliente podem criar, ler ou atualizar arquivos remotamente. Para isso, uma API é disponibilizada a partir de uma biblioteca. A comunicação entre cliente e servidor é feita por chamadas a procedimento remoto (RPC). Um *stub* no lado cliente conecta com o *stub* no lado do servidor e estes permitem a (de)serialização de mensagens. No lado servidor, há um gerenciador de arquivos que, ao receber requisições dos clientes, operam sobre um sistema de arquivos local, buscando arquivos para leitura ou escrevendo novos arquivos no seu dispositivo de armazenamento local.



O lado cliente dispõe ainda de um mecanismo de cache. Dessa forma, caso um cliente tente acessar um arquivo acessado recentemente, este pode ter parte do conteúdo em cache e pode ser aproveitado, economizando comunicação com o servidor.

A API com as funções disponíveis aos clientes deve ser oferecida por uma biblioteca. Assim, qualquer programador pode utilizar do serviço implementado, bastando importar a biblioteca e, a partir dela, criar, ler ou escrever em arquivos. Por simplicidade, este sistema de arquivos não implementa hierarquia de diretórios ou funções mais avançadas sobre arquivos. A seguir são apresentadas as funções disponíveis:

```
int abre(int *descriptor_arquivo, char *nome_arquivo):
```

- A função retorna um descritor de arquivo uma vez passado o nome do arquivo a ser aberto. Caso o arquivo não exista, ele será criado. Caso contrário, o descritor referenciar um arquivo já existente e este poderá sobre modificações ou ser lido;
- o valor de retorno inteiro (int) deve representar códigos de erro, na impossibilidade de execução da operação;

```
int le(int descriptor_arquivo, int posicao, byte *buffer, int tamanho):
```

- `descriptor_arquivo` indica o identificador do descritor ao qual se pretende manipular;
- `posicao` indica a posição inicial do arquivo de onde se pretende ler algum conteúdo;
- `buffer` indica o endereço da variável que receberá o conteúdo da leitura;
- `tamanho` indica o número em bytes a serem lidos na operação (ou seja, o número de bytes a partir da posição `posicao`);
- o valor de retorno inteiro (int) deve representar códigos de erro, na impossibilidade de execução da operação.

```
int escreve(int descriptor_arquivo, int posicao, byte *buffer, int tamanho):
```

- `descriptor_arquivo` indica o identificador do descritor ao qual se pretende manipular;
- `posicao` indica a posição inicial do arquivo onde se pretende escrever algum conteúdo;
- `buffer` indica o endereço da variável que contém o conteúdo a ser escrito no espaço de endereçamento
- `tamanho` indica o número em bytes a serem escritos na operação (ou seja, o número de bytes em `buffer`, a partir da posição `posicao`);
- o valor de retorno inteiro (int) deve representar códigos de erro, na impossibilidade de execução da operação.

```
int fecha(int *descriptor_arquivo):
```

- `descriptor_arquivo` indica o identificador do descritor de arquivo a ser fechado.

Coerência de cache

As operações de leitura e de escrita podem acessar partes do arquivo (blocos) localmente ou remotamente. Por exemplo, uma leitura de regiões previamente carregadas na cache local, retornam diretamente para a função de leitura, sem gerar requisições ao servidor. Para isso, os processos solicitantes devem fazer cópias do(s) bloco(s) de interesse e mantê-la em uma cache local quando da escrita ou leitura desses conteúdos.

No caso de leituras e leituras sucessivas, se o processo tiver uma cópia válida do bloco de interesse, basta ler da sua cache. No caso de escritas, o processo deve atualizar o valor do bloco no arquivo em questão e gerar uma invalidação das cópias daquele bloco em caches de outros processos. Este procedimento é comum na implementação de mecanismos para coerência de cache e chama-se *invalidação na escrita*. É permitida a utilização de outras estratégias para coerência de cache.

Heterogeneidade

O trabalho deve demonstrar o potencial de heterogeneidade no uso da biblioteca. Para isso, além da implementação do servidor e cliente em uma determinada linguagem de programação, será necessário disponibilizar a biblioteca para uma segunda linguagem de programação (ex. servidor em C# e clientes em C# e Java). Para facilitar a comunicação entre processos rodando em diferentes linguagens, será necessário utilizar gRPC.

Configuração

Na inicialização do sistema, é necessário iniciar o servidor de arquivos. Você pode fazer uso de arquivos de configuração para manter informações necessárias para o servidor e clientes. Por exemplo, o endereço e a porta do servidor podem estar disponíveis no arquivo `config.txt`.

O formato de requisições e respostas por solicitação de arquivo fica a critério do grupo, assim como a estratégia para a implementação de cache e informações adicionais necessárias para a invalidação de cache dos usuários do serviço.

Uso de padrões de projeto para computação distribuída

Além de atender os requisitos enunciados, o grupo deverá utilizar algum padrão de projeto na sua solução. O padrão pode ser empregado na implementação de algum elemento já enunciado ou para adicionar alguma funcionalidade extra ao serviço. Por exemplo, padrão que visem limitar o tráfego no servidor de arquivos, que permitam a repetição de operações no caso de falhas ou que visem atender requisitos de disponibilidade ou desempenho. A referência a seguir pode ser útil para a escolha do padrão: <https://learn.microsoft.com/pt-br/azure/architecture/patterns/>

Execução

Para realizar testes sobre o sistema implementado, cada grupo deve implementar clientes usando arquivos compartilhados.

- Deve ser possível ilustrar o uso de clientes em diferentes linguagens de programação
- Deve ser possível observar o uso da cache, com acessos sucessivos a posições de um arquivo em cache
- Deve ser possível observar a estratégia de invalidação de cache.

Entrega

O trabalho consiste em:

1. Implementar o servidor de arquivos remoto, a biblioteca em 2 linguagens de programação diferentes e programas clientes de exemplo, de modo a validar as funcionalidades oferecidas.
2. Breve relatório que indique as principais decisões e estratégias de implementação utilizadas, instruções sobre como compilar e executar o código produzido, além da

justificativa da escolha do padrão de projeto e de exemplos de saídas de execução com diferentes parametrizações. Ao final, discuta quais conclusões você observa, indicando limitações observadas na implementação atual.

O trabalho pode ser realizado em **grupos de até 3 participantes**. O trabalho será apresentado em laboratório.

O código-fonte e relatório devem ser enviados pelo Moodle para análise e avaliação.

Os nomes dos participantes do grupo devem constar no relatório entregue no Moodle. Participantes com nomes não referenciados não serão considerados como membros do grupo.